



ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль

F.CSM101 Программчлалын үндсэн аргууд

Хяналт, түүний хийсвэрлэл

Лекц 6

*Доктор (Ph.D) **Ганбаатарын ГАНБАТ***

2023-2024 Хавар

- Илэрхийлэл, Команд
- Дараалал хяналтын команд
- Бүтэцлэгдсэн программчлал
- Рекурс

- Дэд программ
- Дээд эрэмбийн функц
- Онцгой тохиолдол



Дарааллын хяналт: Нийтлэг хийсвэр машины интерпретатор дээрх зааврын биелэлтийг удирдах

Доод түвшний хэл: Дараалал хяналтыг РС-г өөрчлөх замаар хэрэгжүүлдэг.

Дээд түвшний хэл: Тухайн хэлний бүтэц нь физик машин дээрхтэй харьцуулахад илүү хийсвэр механизмыг хянах, хэрэгжүүлэх боломжтой.

**Илэрхийллийн синтакс ба семантик, Команд, Бүтэцлэгдсэн
болон бүтэцлэгдээгүй хяналт, Рекурс**

Илэрхийлэл (Expression)

- Ихэнх PL-ний суурь бүрдэл хэсэг болдог.
 - Команд болон зарлалттай нягт уялдаатай
 - Командгүй декларатив хэлэнд ч тоон эсвэл симболик илэрхийлэл бий

ТОД 6.1 (Илэрхийлэл): Тооцоолж утга бүтээдэг эсвэл алдаа өгдөг (энд тодорхойгүй/undefined байна) синтакс нэгжийг *илэрхийлэл* гэнэ.

- Командаас ялгаатай нь утга бүтээдэг. Жнь: $4+3*2$.
 - Илэрхийллийн хяналт: Үйлдлийн эрэмбэ шаардлагатай (10 эсвэл 14)
- Илэрхийлэл нь тоон бус байж болно
 - Жнь: Lisp хэлэнд а ба b-ээс бүрдсэн хосыг (cons a b) гэж илэрхийлнэ
- *Үйлдэл (Operator)*: (+, cons ...) болон аргумент (or *операнд*)
 - Операнд (Operand): Тогтмол, хувьсагч ... ;
 - Аргумент нь бас илэрхийлэл байж болно

Программд тэмдэгт дараалал хэлбэрээр шугаман бичлэг хэрэглэдэг

- **Infix**: бинар үйлдлийн тэмдэг нь хоёр операнд илэрхийлэл дунд $(x+y)*z$;
 - *syntactic sugar*: Ada $+(a,b)$; C++ $a.operator+(b)$; Python $a.__add__(b)$
 - Салаа утгагүй байх: Хаалт, үйлдлийн эрэмбийн дүрэм шаардлагатай.
 - Бинар бус операторын хувьд ховор хэрэглэгддэг.
- **Prefix**: (*prefix Polish notation*) Үйлдлийн тэмдэгт нь өмнөө $*(+(a\ b)+(c\ d))$
 - Хаалт хэрэглэхгүй байж болно. Жнь функцийн хувьд $f(a\ b)$ or $f\ a\ b$.
 - Үйлдлийн эрэмбэ хамааралгүй
- **Postfix** (*reverse Polish*): Үйлдлийн тэмдэг нь ардаа $a\ b\ +\ c\ d\ +\ *$
 - Зарим компиляторын завсрын кодонд, мөн PL-д ч (жнь Postscript).
- Олон операндын хувьд Infix-д туслах оператор хэрэгтэй
- Polish нь хаалтыг бүрэн орхиж болно. Уншихад хялбар. 70,80-аад онд, Халаасны тооны машинд амжилттай хэрэглэсэн

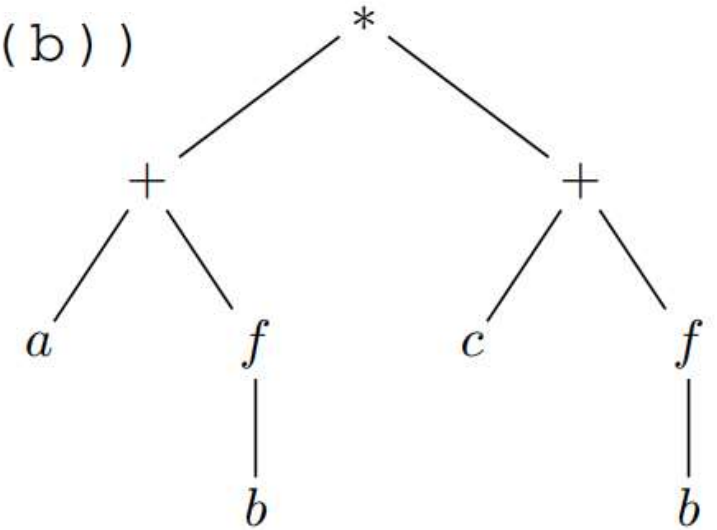
- Үр дүнг олж авах үед утга салаалалт үүсдэг.
 - Илэрхийллийг PL-д ихэвчлэн дотооддоо мод хэрэглэн дүрсэлдэг.
- Хаалтын системтэй хэрэглээ. $4+3*5$ үр дүн нь 35 биш
 - Pascal: $x=4$ and $y=5$. Энд and, = нь логик, харьцуулах оператор
 - Алдаа өгнө: $(x=4)$ and $(y=5)$ биш $x=(4 \text{ and } y)=5$.
 - Хаалтыг бичихгүй гэвэл эрэмбийн дүрэм (precedence rules) шаардлагатай
- Операторын хамаарал. Жнь 15-5-3: $(15-5)-3$ эсвэл $15-(5-3)$
 - Ерөнхийдөө зүүн – баруун байдаг
 - Зэрэгт дэвшүүлэх баруун-зүүн: $5^{(3^2)}$ нь $5^{**}(3^{**}2)$; болон $(5^3)^2$ нь $((5^{**}3)^{**}2)$;
- Эрэмбэ, хамаарлын дүрмийн далд хэрэглээ нь хаалтанд ил хэрэглээтэй давхардан алгоритмын зохиомжийг ихээхэн хүндрүүлдэг.
 - Зүүн-баруун (эсвэл эсрэг) нэг скан хийх боломжгүй

1. Илэрхийлэлээс тэмдэгт уншиж, стект хийнэ;
2. Хэрэв тэмдэгт нь үйлдэл бол C -д аргументынх тоог олгоод 1-рүү шилжинэ.
3. Хэрэв уншсан тэмдэгт нь операнд бол C -г бууруулна.
4. Хэрэв $C \neq 0$ бол 1-рүү шилжинэ.
5. Хэрэв $C = 0$ бол дараах үйлдлүүдийг гүйцэтгэнэ
 - a) Стек хадгалсан сүүлийн операторыг стект яг доор байгаа операндуудад хэрэглэж, үр дүнг R регистрт хадгалж, стекээс оператор болон операндуудыг устгаж, R -ийн утгыг стек дээр хадгална.
 - b) Хэрэв стек дээр операторын тэмдэг байхгүй бол 6 руу шилжинэ.
 - c) C тоологчийг $n - m$ болгоно, энд n нь стек дээрх хамгийн дээд талын операторын аргументийн тоо, m нь энэ операторын дээрх стек дээр байгаа операндуудын тоо юм.
 - d) 4 рүү шилжинэ.
6. Унших дараалал хоосон биш бол 1 рүү шилжинэ

1. Илэрхийлэл дэх дараагийн тэмдгийг уншиж, стект хийнэ.
2. Хэрэв уншсан тэмдэг нь оператор бол түүнийг стек дээрх яг доор нь байгаа операндуудад хэрэглэнэ, үр дүнг R-д хадгалана, стекээс оператор болон операндуудыг гаргаж, R утгыг стек рүү хийнэ.
3. Унших дараалал хоосон биш бол 1 рүү шилжинэ.
4. Хэрэв уншсан тэмдэг нь операнд бол 1 рүү шилжинэ.

$$(a + f(b)) * (c + f(b))$$

ИЛЭРХИЙЛЭЛ Тооцоолол (Evaluation)



- Илэрхийллийн *синтакс мод*:
 - Үйлдлийн эрэмбэ, хамаарал тодорхой (но хаалт).
 - Дэд мод нь операнд тул доороос нь тооцоолох ёстой.
- Компилятор хэрэгжилттэй хэл (*Синтакс шинжилгээ -> Гаргалгааны мод*)
 - Эх кодын infix-ийг мод хэлбэрт хөрвүүлдэг. (Объект код үүсгэхэд хэрэглэнэ. О.код нь компиляц явагдах машинаас хамаарна. Уламжлалт физик машины хувьд засврын үр дүнг хадгалах регистр, санах ой байршил ашигладаг код болгоно. жнь opcode operand1 operand2)
 - Зарим үед prefix, postfix. (стек дээр тооцоолол хийдэг. Жнь: SNOBOL хэл)
- Интерпретатор хэрэгжилттэй хэл
 - Эх код ихэвчлэн infix. Мод хэлбэрт хөрвүүлэхэд тохиромжтой (Мод тойрох аргаар шууд тооцоолно. Жнь Lisp интерпретац, программ бүтнээрээ мод).

- Infix нь
 - Опертандуудын эрэмбийг тодорхойлохгүй: $a+f(b)$ ба $c+f(b)$
 - Операндыг операторын өмнө тооцох эсэх нь тодорхойгүй
- Математикийн хувьд ялгаагүй ч бидний хувьд:
 - **Side effects:** $a-f(b)-c*d$, энд $f(b)$ нь c эсвэл d -г өөрчилж болно
 - **Finite arithmetic:** $b>c>0$, $a=\text{maxInt}$ бол баруун-зүүн $(a-b+c)$ үед хэвийн, харин зүүн-баруун $(a+b-c)$ үед халилт үүснэ.
 - **Undefined operands:** $a==0?b:b/a$, *eager* (эхлээд операнд) vs *lazy evaluation*
 - **Short-circuit evaluation:** $a==0||b/a>2$, энд $a=0$ бол алдаа өгнө
 - **Optimization:** Физик машины байгуулалтаас шалтгаалж илэрхийллийн тооцооллын үр ашигт нөлөөлдөг. a -г хараахан авах боломжгүй байх үед:

```
a = vector[i];  
b = a*a + c*d;
```

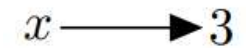
Команд (Concept of Command)

- Илэрхийлэл бүх PL-д байдаг бол команд голдуу императив хэлэнд байдаг

ТОД 6.2 (Команд): Команд нь гадаад нөлөө (side effects) бүхий гүйцэтгэлтэй байж болох синтакс нэгж бөгөөд заавал утга буцаахгүй.

- Команд болон ер нь бусад бүрдлүүд үр дүнд нөлөөлж байгаа ч тухайн контекстэд ямар ч утга буцаахгүй бол гадаад нөлөө өгнө.
 - Print("Hello world");
- Командын гүйцэтгэл нь гадаад нөлөөгөөр *төлөв* өөрчилдөг.
- Төлөв өөрчлөх үндсэн бүрдэл нь *ymга олголт (assignment)* байна.

- *Modifiable variable*: "Өөрчлөх боломжтой сав". Санах ойн байрлал
- *Reference variable*: Хандалтын механизм зөвшөөрдөг. Контейнерыг утга болгон заасан



- Reference загвартай хэл: Python, Scala, and Java (зөвхөн class төрөлдөө).
- (Цэвэр) функциональ хэлний хувьсагчийн ойлголт математиктай ижилт
 - Хувьсагч гэдэг нь утгыг төлөөлсөн нэр.
 - "хувьсагч байдаггүй" гэж ихэвчлэн хэлдэг: Modifiable хувьсагчгүй (эсвэл утгууд нь агуулахад биш reference загвартай).
- Логик хэл нь функциональ хэлтэй адил хувьсагч нь утганд холбосон нэр байна
 - Хувьсагчийн нэр болон утгын холбоос үүссэн бол арилгах боломжгүй.

- “Өөрчлөгдөх хувьсагч” үндсэн команд, ө.х. нэртэй холбосон утгыг (эсвэл агуулгыг) өөрчлөх.

$X := 2$

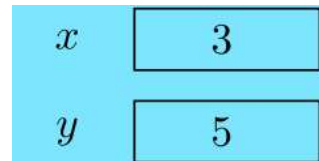
- X -ийн контейнер нь өмнө нь байсан утгын оронд 2 гэсэн утгыг агуулна.
- Үүнээс хойш X -г (“ X -д хандах”) ашигласнаар 2-ын утгыг буцаана.
- Өмнө нь хадгалагдаж байсан утга бүрэн алдагдана.

$X := X + 1$

- l -утгууд: зүүн талд байгаа X нь шинэ утгыг хадгалахад хэрэглэгдэх байршил.
- r -утгууд: баруун талд байгаа X нь савны доторх утгыг илэрхийлнэ.
- Хэрэв хэл нь хувьсагчийн лавлагааны загвартай бол l -утгууд нь байршлыг биш харин лавлагааг илэрхийлнэ.

`exp1 OpAss exp2`

- Өөрчлөгдөх хувьсагчтай үеийн семантик:
 - `exp1`-ийн l -утгыг тооцоолж, контейнер loc -ийг тодорхойлох;
 - `exp2`-ийн r -утгыг тооцоолох, val утгыг авах;
 - loc -ийн агуулгыг өмнө нь байсан val -г орлуулан өөрчлөх.
- Хувьсагчийн лавлагаатай үеийн семантик:
 - `exp1`-ийн l -утгыг тооцоолж, лавлагаа ref -ийг тодорхойлох;
 - `exp2`-ийн r -утгыг тооцоолох, val утгыг авах;
 - лавлагааны ref -г өөрчилнө, ингэснээр одоо val -д хамаарна.
 - (Өөрчлөгдөх хувьсагчийнх шиг val нь хуулагдахгүй).

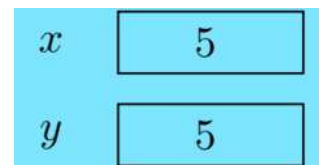


before

$x \longrightarrow 3$

$y \longrightarrow 5$

`x=y;`



after

$x \longrightarrow 5$

$y \longrightarrow 5$

- Утга олголт нь императив ("холимог" декларатив) хэлний үндсэн команд;
 - Анхан шатны тооцооллын алхамыг илэрхийлдэг.
- Бусад ихэнх командууд нь дарааллын хяналтыг тодорхойлдог, өөрөөр хэлбэл утга олголтоор хийгдсэн төлөвийн өөрчлөлтийг гүйцэтгэх дарааллыг зааж өгөхөд ашиглагддаг.
 - Дарааллыг ил хяналтын команд (explicit sequence control commands)
 - Нөхцөлт/Сонгох команд (Conditional/Selection commands)
 - Давталттай команд (iterative commands)

- Дараалсан команд (Sequential Command) `c1 ; c2 ; ... ; cn`
- Нийлмэл команд (Composite Command) Нийлмэл команд нь командын дарааллыг нэгж болгон бүлэглэдэг.

```
begin      {  
...      ...  
end      }
```
- Goto
 - Ассемблэр хэлний `jump`, HW-ийн дараалал хяналтыг шууд дуурайсан
 - Олон удаа, дураараа ашиглавал хянахад маш хүндрэлтэй болно
 - Блок руу үсрэхэд, идэвхжлийн бичлэгийг хэзээ, яаж үүсгэх вэ `goto A`
 - Хэлний илэрхийлэх чадварт нөлөөлгүй, Одоогийн хэлүүд бүрэн холдсон
- Other Sequence Control Commands
 - Goto командын хязгаарлагдмал хувилбар: `break`, `continue`, `case`, `return`
 - Онцгой тохиолдол (exceptions)

- Нөхцөлт болон сонгох команд нь тохирох логик нөхцөл дээр үндэслэн тооцооллын хоёр ба түүнээс дээш боломжит үргэлжлэл дээрх нэг хувилбарыг илэрхийлдэг.

```
if Bexp then C1 else C2
if Bexp then C1
if Bexp1 then C1
    elseif Bexp2 then C2
    ...
    elseif Bexpn then Cn
    else Cn+1
endif
```

```
case Exp of
    label1: C1;
    label2: C2;
    ...
    labeln: Cn;
else Cn+1
```

- Goto-г эс тооцвол энэ хүртэл бидний үзсэн бүх команд нь тооцооллыг илэрхийлэх хязгаарлагдмал боломжтой бөгөөд программ урт байна.
 - Доод түвшний хэлэнд бүх боломжит алгоритмуудыг илэрхийлэх чадварыг jump өгдөг.
 - Дээд түвшний хэлүүдэд goto командаас зайлсхийж хоёр үндсэн механизмыг ашигладаг: *бүтэцлэгдсэн давталт* ба *рекурс*.
- Хэл шинжлэлийн түвшинд давталтыг 2 хуваана
 - *төгсдөлгүй давталт*
 - *төгсдөг давталт*

```
while (Bexp) do C  
repeat C until Bexp  
do C while (Bexp)
```

```
for I = start to end by step do  
    body
```

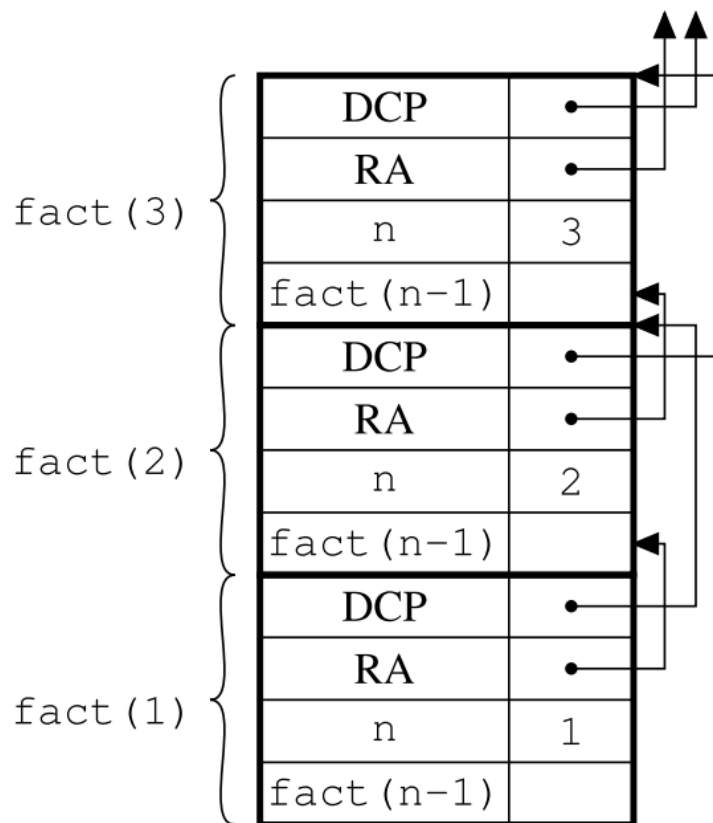
Бүтэцлэгдсэн программчлал

- Дээрээс доош буюу шаталсан программ дизайн
 - Программыг эхний (хийсвэр) тодорхойлолтоос эхлээд алхам бүрт дараалсан нарийвчилсан сайжруулалт нэмэн боловсруулдаг.
- Кодын модульчлал
 - Программд командуудыг процедур, функц, модуль болгон бүлэглэх
- Утга агуулгатай нэр ашиглалт
 - Хувьсагч, процедур зэрэгт агуулгатай нэр ашигласнаар засвар үйлчилгээний аливаа өөрчлөлтийг хийхэд хялбар болгодог.
- Сэтгэгдлийн (Comment) өргөтгөсөн хэрэглээ
 - Сэтгэгдэл нь кодыг ойлгох, тэстэлх, шалгах, залруулах, өөрчлөхөд зайлшгүй шаардлагатай. Сэтгэгдэлгүй программ бол урт болох тусмаа ойлгомжгүй.
- Бүтэцлэгдсэн өгөгдлийн төрлийн ашиглалт
- Бүтэцлэгдсэн хяналтын байгуулалтын ашиглалт
 - Жнь: Нэг оролттой байх, нэг гаралттай байх

Рекурс

- Тьюринг эквивалент програмчлалын хэлний давталтын механизм.
 - Өөрийг дууддаг модуль нь рекурс
 - P-г дуусан Q-г өөр P дууддаг бол шууд бус рекурс (mutual recursion).

Dynamic Chain Pointer (DCP)
Result Address (RA)
n
Intermediate Result: fact (n-1)



PL нь физик машиний хийсвэрлэл. Цаашлаад, илэрхийлэх чадвар нь тухайн хэлний хийсвэрлэлийн механизм дээр үндсэндээ суурилдаг.

Хийсвэрлэх: Ямар нэг зүйлийг далдлах буюу өгөгдлөөс хийсвэрлэн авч тухайн объектийг ойлгогдохуйц тодорхойлно.

Хийсвэрлэлийн механизм нь программ зохиогчид шийдэх ёстой асуудлын хүндрэлийг **үнэн зөв, энгийн, ойлгомжтой** байдлаар тайлбарлах гол хэрэгсэл юм.

- Аливаа комплекс программ олон бүрдэл хэсэгтэй байх бөгөөд тэд глобал шийдлийн нэг хэсгийг хангахад үйлчилдэг.
- Асуудлыг дэд асуудал болгон задлах нь
 - Хүндрэлийг илүү сайн удирдах боломжийг олгодог.
 - Илүү хязгаарлагдмал асуудал нь шийдэхэд илүү хялбар.
 - Дэд шийдлүүдийн зохистой нэгдлээр глобал асуудлын шийдлийг бүрдүүлдэг.
- Орчин үеийн бүх дэд програм, процедур, функц бичих боломжтой.
 - *Функц* нь нэрээр ялгагдсан кодын хэсэг бөгөөд өөрийн гэсэн локал орчинтой, бусад кодтой *параметр* ашиглан мэдээлэл солилцох боломжтой.
 - Функцын *тодорхойлолт* (эсвэл зарлалт) ба түүний *хэрэглээ* (эсвэл call) гэсэн хоёр өөр хэл шинжлэлийн механизм болдог.
 - Функц нь *параметр*, *буцах утга*, *нон-локал орчин* гэсэн гурван үндсэн механизмыг ашиглан програмын бусад хэсгүүдтэй мэдээлэл солилцдог.

- Прагматик үүднээс авч үзвэл дэд программууд нь SW зохион бүтээгчид функциональ хийсвэрлэлийг бий болгодог механизм.
 - SW-ийн компонент нь орчинд үйлчилгээ үзүүлдэг бүтэц.
 - Компонентын клиент нь үйлчилгээг хэрхэн бүтээхийг сонирхдоггүй, зөвхөн хэрхэн ашиглахыг сонирхдог.
 - Компонент бүртэй функцийг холбоход клиент мэдэх шаардлагатай зүйлийг мэдэх шаардлагагүй зүйлээс салгах боломжийг олгодог.
 - Клиент нь функцийн их биеээс хамаарахгүй, зөвхөн толгой хэсгээс хамаарч байвал жинхэнэ функциональ хийсвэрлэлтэй болдог.
- Программист эдгээр функцийг зөв ашиглах шаардлагатай,
 - тухайлбал функц болон дуудагч хоорондын харилцан үйлчлэлийг зөвхөн параметр дамжуулах замаар хязгаарлах,
 - дуудлагч болон дуудагдагч хоёрын хооронд мэдээлэл солилцохын тулд нон-локал орчныг ашиглах нь функциональ хийсвэрлэлийг устгана.

- *Параметр дамжуулах журам*: Бодит параметруудийг формал параметруудтэй хослуулах горим ба үүнээс үүсэх семантик
- Дэд програмд параметрийг гурав ангилж болно
 - Input parameters.
 - Output parameters.
 - Input/output parameters.
- Хэрэгжүүлэх тусгайлсан техник нь "дуудалтын горим"-ыг бүрдүүлдэг.

Call by Value, by Reference, by Constant, by Result, by Value-Result, by Name

- Ямар төрлийн communication зөвшөөрдөг.
- Бодит параметрийн ямар хэлбэрийг зөвшөөрдөг.
- Горимын семантик.
- Каноник хэрэгжилт.
- Түүний өртөг.

ХЯНАЛТЫН ХЙИСВЭРЛЭЛ >> Өндөр эрэмбийн функц

- Функц нь өөр функцийг параметрт авах эсвэл буцаавал илүү *өндөр эрэмбэтэй (high-order)* байна.

```
{int x = 1;
int f(int y){
    return x+y;
}
void g (int h(int b)){
    int x = 2;
    return h(3) + x;
}
...
{int x = 4;
  int z = g(f);
}
```

Functional parameters

```
{int x = 1;
void->int F () {
    int g () {
        return x+1;
    }
    return g;
}
void->int gg = F();
int z = gg();
}
```

Functions as results

- Хуучны императив хэлэнд дэд программ нь үндсэн утгаас өөр статустай.
 - Жнь Си хэл, зөвхөн функц тодорхойлолтоор зарлах ба функцэд нэр өгнө.
 - Бүхэл тоон утгын хувьд *нэргүй* илэрхийлэлтэй.
- Орчин үеийн хэлний хувьд, Функц нь first-class байдаг.
 - Функцийг нэргүй буюу *ламбда илэрхийлэлээр* дүрсэлдэг.

```
def sq(x):  
    return x*x
```

Scala

```
(x: Double) => x * x
```

Python

```
lambda x: x * x  
  
(lambda x: x * x)(2.0)  
  
sq = lambda x: x * x
```

- Онцгой тохиолдлыг программ биелэх явцад шалгадаг бөгөөд хяналтын ердийн урсгалд зохицуулагдах ёсгүй (эсвэл боломжгүй) эвент.
 - Динамик семантик алдаа (0-д хуваах, орон халилт ...) гарсан эсэх
 - Одоогийн тооцооллыг зогсоож, хяналтыг программын өөр цэг рүү шилжүүлэх нөхцөл үүссэн эсэх
- Ерөнхийдөө, онцгой тохиолдлыг зөв удирдахын тулд хэл нь:
 1. *Ямар* онцгой тохиолдлуудыг барих, хэрхэн тодорхойлохыг заасан.
 2. Тооцооллын онцгой байдлаар зогсоход хүргэдэг механизмын онцгой тохиолдол хэрхэн *гарч болохыг* заасан.
 3. Онцгой тохиолдлыг хэрхэн боловсруулах, жнь онцгой тохиолдол гарах үед ямар үйлдэл хийх, биелэлтийн хяналтыг хааш нь шилжүүлэхийг заасан.



ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль

АНХААРАЛ ТАВЬСАНД БАЯРЛАЛАА